

Cartesian 경로 재생(replay) 방법 비교 — twist vs pose vs joint

녹화된 로봇 동작을 다시 재생할 때, 무엇을 명령(command)하느냐에 따라 재현 충실도가 크게 달라진다. 이 문서는 "녹화된 EE(end-effector) 경로를 드리프트 없이 재생하려면 어떻게 해야 하는가"를 정리한다.

핵심 원리: 위치 명령은 페루프, 속도 명령은 개루프

- **twist(속도) 명령** — Servo가 $\dot{q} = J^{-1} \cdot v$ 를 적분해 위치를 만든다. 매 주기의 명령이 "얼마나 빨리"이므로, 오차를 되돌릴 기준 위치가 없다. 미세 오차(노이즈, 이산화, 타이밍, Jacobian 불일치)가 시간에 따라 **누적(drift)**된다.
- **pose(위치) 명령** — 매 주기 $error = target_pose - current_pose$ 를 계산해 그 오차를 줄이는 방향으로 움직인다. 목표 위치 자체가 기준이므로 **오차가 스스로 교정**된다 → 드리프트 없음.

즉 되먹임(feedback) 대상이 "속도"냐 "위치"냐의 차이이다.

방법 A) Servo Pose Tracking (실시간 페루프)

Movelt Servo에는 velocity(delta_twist) 모드 외에 **pose tracking 모드** (moveit_servo의 PoseTracking)가 있다. 목표 PoseStamped를 토픽으로 흘려주면 Servo가 다음을 수행한다.

매 제어주기: $error = target_pose - current_ee_pose(FK)$
 $v_cmd = Kp_lin \cdot error_pos + Kp_ang \cdot error_rot$ (PID)
 $\dot{q} = J^{-1} \cdot v_cmd \rightarrow$ 컨트롤러로

- 녹화된 EE pose 스트림을 순서대로 target_pose로 발행하면 각 pose를 추종한다.
- **페루프**라 pose 오차를 계속 교정 → 장시간에도 드리프트 없음.

특징 / 한계

- 실시간·반응형이라 목표가 계속 바뀌어도 된다(온라인 replay에 적합).
- PID 게인 튜닝이 필요하다. 빠른 구간에선 **정상상태 lag**(목표보다 살짝 뒤처짐)이 생길 수 있다.
- 여전히 6-DOF 명령이라 **팔꿈치(여유자유도) 형상은 원본과 다를 수 있다** — Servo가 null-space를 자체적으로 푼다.
- 특이점 / 관절한계 근처에서 속도 스케일다운·halt가 일어난다.

repo 상태: /moveit_servo 노드는 이미 런치에서 뜬다. delta_twist 대신 pose-tracking 인터페이스를 쓰도록 구성만 하면 된다.

적합한 경우: 녹화 pose 가 스트림으로 계속 들어오고, 실시간으로 따라가게 하고 싶을 때.

방법 B) Moveit Cartesian Path (계획 후 실행)

녹화된 EE pose 들을 **waypoint 리스트** 로 넘겨 computeCartesianPath 로 직교공간 직선보간 궤적을 미리 생성한 뒤, JointTrajectoryController 로 실행한다.

녹화 EE poses → computeCartesianPath(waypoints) → 관절 궤적(q(t)) 생성
→ panda_arm_controller 로 실행 (관절 페루프 위치제어)

- 실행이 **관절 위치제어** 라 정확하고 드리프트 없음. 궤적을 미리 시간 파라미터화(속도 / 가속 제한 반영)한다.

특징 / 한계

- **plan-then-execute** (반응형 아님). 알려진 녹화 경로 전체를 한 번에 계획·실행하는데 최적이다.
- waypoint 간격이 너무 크거나 특이점 부근이면 computeCartesianPath 가 **$\text{fraction} < 1.0$** (경로 일부만 생성)으로 실패할 수 있다 → 녹화 pose 를 촘촘히 주게 좋다.
- IK 를 waypoint 마다 풀어 관절해를 정하므로 **팔꿈치 형상은 원본과 다를 수 있다** (유효하고 일관된 해이긴 하다).

repo 상태: 이미 API 가 있다 — src/rdfp/rdfp/moveit/move_group_client.py 의 follow_trajectory(waypoints: list[Pose]) / follow_trajectory_async / plan_trajectory ([CLAUDE.md](#) 에도 "cartesian planning + execution" 으로 명시). 녹화 pose 를 Pose 리스트로 만들어 넘기면 그대로 동작한다.

적합한 경우: 녹화된 Cartesian 경로가 이미 확보돼 있고, 부드럽고 검증된 한 번의 replay 를 원할 때.

세 방법 비교

방식	명령 종류	루프	드리프트	관절(팔꿈치) 재현	성격	repo 지원
delta_twist (twist) 속도		개루프 적분	❌ 누적	❌	실시간	EeTwistPublisher (모니터링용)
Servo pose tracking	위치	페루프	✅ 없음	❌	실시간·반응형	/moveit_servo 재구성

방식	명령 종류	루프	드리프트	관절(팔꿈치) 재현	성격	repo 지원
Cartesian path	위치(궤적)	폐루프	✓ 없음	✗	계획 후 실행	MoveGroupClient.follow_trajectory
(참고) joint replay	관절 위치	폐루프	✓ 없음	✓ 정확	계획 / 재생	target_joint_states_executor

요약 및 권장

- **Cartesian** 경로만 충실히 재생 → A(실시간 스트림이면) 또는 B(경로 확보돼 있으면). 둘 다 위치 폐루프라 드리프트 없음.
- 다만 A, B 모두 EE 경로만 맞추고 팔꿈치(여유자유도)까지는 원본과 다를 수 있다. 팔꿈치 형상까지 정확히 재현하려면 결국 **joint 위치 replay** 가 유일하다.
- 실용적으로는 이 repo 에 이미 있는 **B(MoveGroupClient.follow_trajectory)** 가 가장 손쉽게 검증 가능한 출발점이다.

참고: 왜 twist → delta_twist_cmds 는 replay 에 부적합한가

녹화된 joint_states 를 다시 읽어 EeTwistPublisher 로 twist 를 만들고

/delta_twist_cmds 에 발행하는 방식은 다음 이유로 원래 동작을 충실히 재현하지 못한다.

1. **적분 드리프트** — 속도 명령만 주면 위치 오차를 되돌릴 기준이 없어 오차가 시간에 따라 누적된다(원리적 한계).
2. **절대 위치 부재** — twist 는 "얼마나 빨리" 만 담고 "어디에" 는 담지 않는다. 시작 pose 가 다르면 전체 재생이 오프셋된다.
3. **여유자유도(7-DOF) 미재현** — EE twist 는 6-DOF 라 팔꿈치 null-space 가 원본과 달라진다.
4. **특이점 / 한계 스케일링, 타이밍, 미분 노이즈** — 모두 오차로 적분되어 lag·이탈을 만든다.

따라서 EeTwistPublisher 의 twist 는 **모니터링 / 학습 데이터(관측 EE 속도)** 용으로 쓰고, 로봇을 움직이는 replay 명령 소스로는 사용하지 않는다.

관련 문서

- src/rdfp/rdfp/moveit/move_group_client.py — follow_trajectory / plan_trajectory (Cartesian path).
- docs/moveit/move_group_client_programmers_guide.md
- docs/moveit/servo_client_programmers_guide.md

- `src/rdfp/rdfp/moveit/target_joint_states_executor.py` — joint 위치 replay 경로.
- `src/rdfp/launch/replay_panda_mock.launch.py` — replay 전용 런치 스택.